GATE DA 2027

Artificial Intelligence

Search, logic and reasoning under uncertainty with the stories of Ramu and Somu

Ramu and Somu once more.
Ramu asks "but why?" about everything. **Somu** answers with a maze, a chess board, a detective's notebook and a rainy-day sprinkler.

Syllabus covered (GATE DA, Section 7): *search:* uninformed (BFS, DFS, depth-limited, iterative deepening, uniform-cost), informed (greedy best-first, A*, heuristics) and adversarial (minimax, alpha–beta pruning); *logic:* propositional logic and predicate (first-order) logic; *reasoning under uncertainty:* conditional independence and Bayesian networks, exact inference by variable elimination, approximate inference by sampling.

Every trace, truth-table count, unification result and probability in the worked examples was computed by code.

Contents

1 Problem Solving by Search **3**

2 Uninformed (Blind) Search **3**

3 Informed (Heuristic) Search **5**

3.1 Heuristics for the 8-puzzle 5

4 Adversarial Search: Games **6**

5 Propositional Logic **8**

5.1 Syntax and semantics 8

5.2 Normal forms 9

5.3 Inference algorithms 9

6 Predicate (First-order) Logic **10**

6.1 Unification and resolution in first-order logic 11

7 Reasoning under Uncertainty **12**

7.1 Probability toolkit 12
7.2 Bayesian networks 12
7.3 Conditional independence and d-separation 13
7.4 Exact inference 14
7.5 Approximate inference by sampling 14

8 One-page Cheat Sheet **15**

9 Practice Questions **15**

1 Problem Solving by Search

Ramu & Somu: Finding the way out of a maze

Ramu is in a maze and wants the exit. Somu: "Write the maze down as a puzzle with four parts: where you *start*, what *moves* you may make from each place, a test for *am I at the exit?*, and the *cost* of each move. Then the question is only: in what order should you try the moves?"

Formulating a search problem

- **State space**: all configurations; **initial state**; **actions** available in each state; **transition model** (result of an action); **goal test**; **path cost** (sum of step costs $c(s, a, s') \geq 0$).
- A **solution** is a path from the initial state to a goal; an **optimal** solution has the least cost.
- **Search tree**: root = initial state, children = successors. **Frontier** (open list) = generated but not yet expanded nodes. **Graph search** also keeps an **explored** set to avoid repeating states (essential when the space has cycles).
- Parameters: **branching factor** b, depth of the shallowest goal d, maximum depth m.
- Strategies are compared by: **completeness** (finds a solution if one exists?), **optimality**, **time** and **space** (counted in nodes).

2 Uninformed (Blind) Search

Ramu & Somu: Searching without a map

"Uninformed" means the algorithm knows only the problem's rules, nothing about *which direction looks promising*. Somu compares four explorers: one explores the maze level by level (**BFS**), one runs down a corridor to its end before backing up (**DFS**), one limits how far he will run (**depth-limited**) and keeps raising the limit (**iterative deepening**), and one always extends the cheapest path so far (**uniform-cost**).

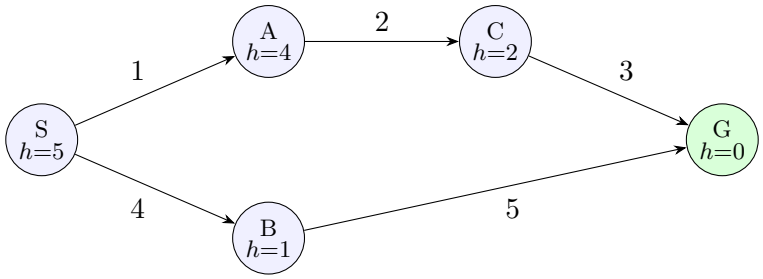
The strategies

- **Breadth-first search (BFS)**: frontier is a **FIFO queue**; expands all nodes at depth k before depth $k + 1$. Finds the shallowest goal.
- **Depth-first search (DFS)**: frontier is a **LIFO stack** (or recursion); goes deep first, backtracks on dead ends.
- **Depth-limited search (DLS)**: DFS with a depth cutoff ℓ (nodes at depth ℓ have no successors).
- **Iterative-deepening search (IDS)**: run DLS for $\ell = 0, 1, 2, \ldots$ until a goal is found. Combines DFS's small memory with BFS's completeness and shallowest-goal property.
- **Uniform-cost search (UCS)**: frontier is a **priority queue** ordered by path cost $g(n)$; expands the cheapest node (Dijkstra's algorithm). The goal test is applied when a node is *expanded*, not when generated.
- **Bidirectional search**: search from start and goal at once; meet in the middle.

Strategy	Complete?	Optimal?	Time	Space
BFS	yes (b finite)	yes if step costs equal	$O(b^d)$	$O(b^d)$
DFS	no (loops / infinite depth)	no	$O(b^m)$	$O(bm)$
Depth-limited	no unless $\ell \geq d$	no	$O(b^\ell)$	$O(b\ell)$
Iterative deepening	yes	yes if step costs equal	$O(b^d)$	$O(bd)$
Uniform-cost	yes if all costs $\geq \varepsilon > 0$	yes	$O\big(b^{1+\lfloor C^*/\varepsilon \rfloor}\big)$	same
Bidirectional	yes	yes (unit cost)	$O(b^{d/2})$	$O(b^{d/2})$

Counting nodes

BFS generates $1 + b + b^2 + \cdots + b^d = \dfrac{b^{d+1} - 1}{b - 1}$ nodes up to depth d. IDS re-generates shallow levels but this is cheap, because most nodes are in the deepest level: nodes generated $= \sum_{i=0}^{d} (d + 1 - i)\, b^i$.
For $b = 10$, $d = 5$: BFS $= 111\,111$; IDS $= 6 + 50 + 400 + 3000 + 20\,000 + 100\,000 =$ **123 456**, only 11% more. For $b = 3$, $d = 4$: BFS $= 121$, IDS $= 179$.



One graph, five algorithms (start S, goal G; neighbours tried in the order A then B)

Step costs are on the edges; h values are used later. Goal-test on expansion.

Algorithm	Expansion order	Path found	Cost
BFS	S, A, B, C, G	S–B–G (fewest edges)	9
DFS	S, A, C, G	S–A–C–G	6
UCS	S(0), A(1), C(3), B(4), G(6)	S–A–C–G	6 (optimal)

BFS finds the path with the fewest *edges* but not the cheapest. UCS orders by g: it first reaches G through B at cost 9, but a cheaper copy of G ($g = 6$ through C) is pushed before G is expanded, so it returns 6.

Common GATE traps

(1) BFS is optimal only for *equal* step costs. (2) DFS (tree search) can loop forever on cycles; graph search with an explored set fixes this but not infinite depth. (3) IDS is not wasteful: the repeated work is dominated by the last level. (4) UCS applies the goal test on *expansion*. (5) The space of BFS is its major weakness (b^d nodes in memory), whereas DFS needs only $O(bm)$.

3 Informed (Heuristic) Search

Ramu & Somu: A compass in the maze

Now Somu gives Ramu a compass that always shows the straight-line distance to the exit. This is a **heuristic** $h(n)$: an *estimate* of the cost still to go from node n to a goal. It is not exact, but it tells Ramu which corridors look promising.

Greedy best-first and A*

- **Greedy best-first**: expand the node with the smallest $h(n)$. Fast when lucky, but **not optimal**, and incomplete in tree search.
- **A* search**: expand the node with the smallest $f(n) = g(n) + h(n)$, where $g(n)$ is the cost so far and $h(n)$ the estimated cost to go. So f estimates the cost of the cheapest solution *through* n.
- $h \equiv 0$ gives uniform-cost search; $g \equiv 0$ gives greedy search; unit costs and $h \equiv 0$ give BFS.

Admissible and consistent heuristics

Let $h^*(n)$ be the true cheapest cost from n to a goal.

- **Admissible**: $0 \le h(n) \le h^*(n)$ for all n (never overestimates; "optimistic").
- **Consistent (monotone)**: $h(n) \le c(n, a, n') + h(n')$ for every edge, and $h(\text{goal}) = 0$ (a triangle inequality).
- **Consistent $\Rightarrow$ admissible** (not conversely).
- **Theorem:** A* *tree* search is optimal with an admissible h; A* *graph* search is optimal with a consistent h. With a consistent h, f never decreases along a path and each node is expanded at most once with its optimal g.
- A* is complete (finite b, costs $\ge \varepsilon$), optimal, and *optimally efficient* for a given heuristic, but its memory use is exponential ($O(b^d)$ in bad cases).
- **Dominance:** if $h_2(n) \ge h_1(n)$ for all n (both admissible), h_2 dominates h_1 and A* with h_2 expands no more nodes. Admissible heuristics can be derived from **relaxed problems**; the maximum of two admissible heuristics is admissible.

A* on the graph above

Check: $h^*(S) = 6$, $h^*(A) = 5$, $h^*(B) = 5$, $h^*(C) = 3$, and $h = (5, 4, 1, 2)$ never exceeds them, so h is admissible. Consistency: $S \to A$: $5 \le 1 + 4$; $S \to B$: $5 \le 4 + 1$; $A \to C$: $4 \le 2 + 2$; $B \to G$: $1 \le 5$; $C \to G$: $2 \le 3$. All hold, so h is consistent.
A* expansion (ties broken by insertion order): $S\,(f = 5)$, $A\,(f = 1 + 4 = 5)$, $B\,(f = 4 + 1 = 5)$, $C\,(f = 3 + 2 = 5)$, $G\,(f = 6 + 0 = 6)$. Result: path $S{-}A{-}C{-}G$, cost **6** (optimal).
Greedy best-first expands S, then B (smallest $h = 1$), then G: path $S{-}B{-}G$ with cost **9**, *not* optimal.

3.1 Heuristics for the 8-puzzle

Two classic admissible heuristics

h_1 = number of **misplaced tiles** (not counting the blank). h_2 = sum of the **Manhattan distances** of the tiles from their goal squares. Both are admissible (each move shifts one tile by one square) and $h_2 \ge h_1$, so h_2 dominates.

Start $\begin{smallmatrix}7&2&4\\5&&6\\8&3&1\end{smallmatrix}$, **goal** $\begin{smallmatrix}1&2&3\\4&5&6\\7&8&\end{smallmatrix}$

Misplaced tiles: $7, 4, 5, 8, 3, 1$, so $h_1 = 6$. Manhattan distances: tile 7: 4, 2: 0, 4: 3, 5: 1, 6: 0, 8: 1, 3: 3, 1: 2, sum $h_2 = 14$. The true optimal solution has 20 moves (found by BFS), so $6 \le 14 \le 20$ as admissibility requires.
A smaller instance $\begin{smallmatrix}1&2&3\\4&&6\\7&5&8\end{smallmatrix}$ needs 2 moves; $h_1 = h_2 = 2$ (perfect).

Variants

IDA*: iterative deepening on the f-limit (small memory). **Weighted A***: $f = g + w\,h$ with $w > 1$: faster, solution at most w times optimal. **Local search** (hill climbing, simulated annealing) keeps only the current state and is used when the path does not matter; it can get stuck in local maxima.

Common GATE traps

(1) Admissible does not mean consistent. (2) An overestimating heuristic can make A* return a non-optimal path. (3) $f = g + h$, not h alone (that is greedy). (4) Graph-search A* needs a *consistent* heuristic to stay optimal (or reopening of closed nodes). (5) The heuristic must be 0 at goal states. (6) The larger admissible heuristic is better (dominates).

4 Adversarial Search: Games

Ramu & Somu: Chess with a clever opponent

Ramu plays Somu at a two-player board game. Ramu wants to *maximise* his score, Somu wants to *minimise* Ramu's score (a **zero-sum** game). "When I plan a move," says Ramu, "I must assume Somu will reply with the move that is worst for me, then I reply with my best, and so on." That backward reasoning through the game tree is **minimax**.

Minimax

Two players **MAX** and **MIN** alternate; leaves carry utility values (from MAX's point of view). The minimax value of a node is

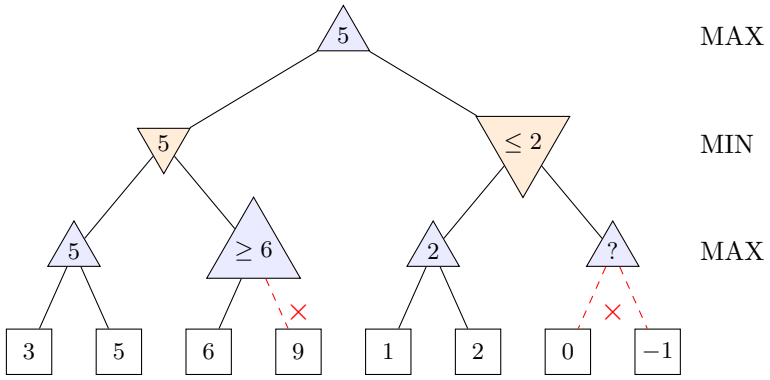
$$\text{MINIMAX}(n) = \begin{cases} \text{utility}(n) & \text{leaf} \\ \max_c \text{MINIMAX}(c) & n \text{ is a MAX node} \\ \min_c \text{MINIMAX}(c) & n \text{ is a MIN node.} \end{cases}$$

Evaluated by depth-first recursion: time $O(b^m)$, space $O(bm)$. It is optimal against an optimal opponent and complete on a finite tree. In real games the tree is cut at a depth limit and non-terminal positions get an **evaluation function**.

Alpha–beta pruning

Carry two bounds down the tree: α = the best (highest) value MAX can already guarantee, β = the best (lowest) value MIN can already guarantee. **Prune** the remaining children of a node as soon as $\alpha \ge \beta$. Alpha–beta returns *exactly the same minimax value*, only faster. Pruning depends on move ordering: with the best ordering the time is $O(b^{m/2})$ (it can search twice as deep in the same time); with random ordering about $O(b^{3m/4})$; worst ordering gives no savings,

$O(b^m)$.



Minimax and alpha–beta on the tree above (leaves left to right $3, 5, 6, 9, 1, 2, 0, -1$)

Minimax: the four MAX nodes give $5, 9, 2, 0$; the two MIN nodes give $\min(5, 9) = 5$ and $\min(2, 0) = 0$; the root gives $\max(5, 0) = \mathbf{5}$.
Alpha–beta: after the first MAX node (value 5), the MIN node has $\beta \le 5$. At the second MAX node the first leaf $6 \ge \beta = 5$ already, so MIN will never choose this branch: the leaf 9 is **pruned**. The MIN node returns 5, so the root has $\alpha = 5$. In the right subtree the first MAX node gives $2 \le \alpha = 5$, so the right MIN node can be at most 2, never beating 5: the whole subtree $(0, -1)$ is **pruned**. Leaves evaluated: $3, 5, 6, 1, 2$ (**5 of 8**); the answer is the same, 5.
Ordering matters: listing the leaves as $[9, 6], [5, 3]$ first forces 6 leaves to be examined for the same tree.

A MAX–MIN tree with three leaves per MIN node

MIN nodes with leaves $(3, 12, 8)$, $(2, 4, 6)$, $(14, 5, 2)$: values 3, 2, 2; root $= \max = 3$. Alpha–beta left to right evaluates $3, 12, 8, 2, 14, 5, 2$: leaves 4 and 6 are pruned (after reading $2 \le \alpha = 3$), but $14, 5, 2$ must all be read, because 14 and 5 are above α. Seven of nine leaves are evaluated.

Other game ideas

Expectimax: chance nodes take the *expected* value of their children (dice, random opponents): e.g. a chance node with outcomes 3 and 7 at probability $\frac{1}{2}$ each has value 5. Alpha–beta cannot prune as freely there. **Evaluation function**: a weighted sum of features (material, position). **Iterative deepening** is used with time limits.

Common GATE traps

(1) Pruning happens when $\alpha \ge \beta$ (including equality). (2) Alpha–beta never changes the root value. (3) Minimax assumes a *perfect* opponent. (4) The number of pruned nodes depends on the ordering of children. (5) Be careful with the sign convention: leaves are values for MAX.

5 Propositional Logic

Ramu & Somu: The detective's notebook

A detective writes facts as short sentences that are either true or false: "the butler was in the kitchen" (B), "the knife is missing" (K). He combines them with "and", "or", "not", "if… then" and asks: *given everything I know, must it follow that the cook is guilty?* That is **propositional logic**: a precise language for combining true/false statements, plus rules to decide what follows from what.

5.1 Syntax and semantics

Connectives and truth tables

Propositions $p, q, \ldots$ (atoms); connectives $\neg$ (not), $\wedge$ (and), $\vee$ (or), $\rightarrow$ (implies), $\leftrightarrow$ (iff). Precedence, high to low: $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$.

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$
T	T	F	T	T	T	T
T	F	F	F	T	**F**	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

$p \rightarrow q$ is false *only* when p is true and q is false (a false premise makes the implication vacuously true). A **model** (interpretation) assigns T/F to every atom; with n atoms there are 2^n models (truth-table rows) and 2^{2^n} distinct Boolean functions (for $n = 2$: 16).

Key notions

- **Valid (tautology)**: true in every model. **Satisfiable**: true in at least one model. **Unsatisfiable (contradiction)**: true in none. α is valid iff $\neg\alpha$ is unsatisfiable.
- **Entailment** $KB \models \alpha$: every model of KB is a model of α. **Refutation theorem:** $KB \models \alpha$ iff $KB \wedge \neg\alpha$ is unsatisfiable.
- **Equivalent** $\alpha \equiv \beta$: same truth value in all models. **Deduction theorem:** $KB \models \alpha \rightarrow \beta$ iff $KB \wedge \alpha \models \beta$.
- An inference procedure is **sound** if it derives only entailed sentences, and **complete** if it derives all entailed sentences.

Equivalences to know

Implication	$p \rightarrow q \equiv \neg p \vee q$
Contrapositive	$p \rightarrow q \equiv \neg q \rightarrow \neg p$ (*not* the converse $q \rightarrow p$, nor the inverse $\neg p \rightarrow \neg q$)
Biconditional	$p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)$
De Morgan	$\neg(p \wedge q) \equiv \neg p \vee \neg q$; $\neg(p \vee q) \equiv \neg p \wedge \neg q$
Double negation	$\neg\neg p \equiv p$
Distribution	$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$; $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$
Negated implication	$\neg(p \rightarrow q) \equiv p \wedge \neg q$
Exclusive or	$p \oplus q \equiv (p \vee q) \wedge \neg(p \wedge q)$

Inference rules: modus ponens $\frac{p,\ p \rightarrow q}{q}$; modus tollens $\frac{\neg q,\ p \rightarrow q}{\neg p}$; hypothetical syllogism $\frac{p \rightarrow q,\ q \rightarrow r}{p \rightarrow r}$;

disjunctive syllogism $\frac{p\vee q,\ \neg p}{q}$; resolution $\frac{p\vee q,\ \neg p\vee r}{q\vee r}$.

Checking by truth table (all verified by program)

- $((p \to q) \wedge p) \to q$ is a **tautology** (modus ponens).
- $(p \to q) \to (q \to p)$ is **not** a tautology (false when p is false and q is true).
- $(p \vee q) \wedge \neg p \wedge \neg q$ is a **contradiction** (0 models).
- Number of models over $\{p, q, r\}$: $p \vee q \vee r$ has 7; $p \wedge q \wedge r$ has 1; $(p \vee q) \wedge (\neg p \vee r)$ has **4**; $p \to (q \to r)$ has 7 (it is false only for $p = q = T, r = F$) and equals $(p \wedge q) \to r$. XOR and $\leftrightarrow$ have 2 each over two atoms.
- $\{p \to q,\ q \to r\} \models p \to r$ holds (checked over all 8 models).

5.2 Normal forms

CNF and DNF

A **literal** is an atom or its negation. **CNF**: a conjunction of clauses, each clause a disjunction of literals, $(l_1 \vee l_2) \wedge (l_3) \wedge \cdots$. **DNF**: a disjunction of conjunctions. Every formula has an equivalent CNF and DNF. **Conversion to CNF:** (1) eliminate $\leftrightarrow$ and $\to$; (2) push $\neg$ inward with De Morgan; (3) distribute $\vee$ over $\wedge$. Example: $p \leftrightarrow q \ \equiv\ (\neg p \vee q) \wedge (\neg q \vee p)$.
Horn clause: at most one positive literal. $p_1 \wedge p_2 \wedge \cdots \wedge p_k \to q$ is the definite-clause form $\neg p_1 \vee \cdots \vee \neg p_k \vee q$.

5.3 Inference algorithms

Resolution refutation

Resolution is **sound and refutation-complete** for CNF: to prove $KB \models \alpha$, add $\neg\alpha$ to the clauses and repeatedly resolve pairs of clauses containing complementary literals; deriving the **empty clause** $\square$ proves the entailment. If no new clauses appear and $\square$ was not derived, $KB \not\models \alpha$.

$KB = \{A \vee B,\ \neg A \vee C,\ \neg B \vee C\} \models C$?

Add the negated goal $\neg C$.

1. $\neg A \vee C$ and $\neg C$ give $\neg A$. 2. $\neg B \vee C$ and $\neg C$ give $\neg B$.
2. $A \vee B$ and $\neg A$ give B. 4. B and $\neg B$ give $\square$.

So $KB \models C$. Confirmed by truth table: the only models of KB are $(A, B, C) = (T, T, T), (T, F, T), (F, T, T)$, and C is true in all three.

Forward chaining, backward chaining, DPLL

- **Forward chaining** (data-driven) for definite clauses: start with the known facts; fire any rule whose premises are all known and add its conclusion; stop when the query is derived or nothing changes. Sound, complete for Horn clauses, runs in time *linear* in the size of the KB.
- **Backward chaining** (goal-driven): start from the query and work back to facts needed to prove it (like Prolog).

- **DPLL** (SAT solver): backtracking over assignments with **unit propagation** (a clause with one unassigned literal forces it) and **pure-literal elimination**. SAT is NP-complete in general; 2-SAT and Horn-SAT are polynomial.

Forward chaining

Rules: $A \wedge B \to C$, $C \to D$, $D \wedge E \to F$. Facts: A, B. Fire rule 1: derive C. Fire rule 2: derive D. Rule 3 needs E, which is unknown. Result: $\{A, B, C, D\}$; the query F is **not** entailed.

Common GATE traps

(1) $p \to q$ is true when p is false. (2) The converse and inverse of an implication are *not* equivalent to it; the contrapositive is. (3) $\neg(p \wedge q) \neq \neg p \wedge \neg q$ (De Morgan flips the connective). (4) $KB \models \alpha$ is about *all* models of KB, not a single one. (5) Resolution proves by contradiction: remember to add $\neg\alpha$. (6) A formula can be satisfiable without being valid.

6 Predicate (First-order) Logic

Ramu & Somu: When "and, or, not" is not enough

Ramu wants to say "every student in the class is clever". With one proposition per student he would need a hundred symbols. Somu invents **variables** and **quantifiers**: $\forall x\,(\text{Student}(x) \to \text{Clever}(x))$. Now one sentence talks about *everything* at once, and we can also say "*some* student scored full marks": $\exists x\,(\text{Student}(x) \wedge \text{Full}(x))$. This richer language is **predicate (first-order) logic**.

Syntax

- **Constants** (Ramu, 0), **variables** (x, y), **function symbols** (fatherOf(x), $x + y$) build **terms**. **Predicates** (Loves(x, y), $x < y$) applied to terms give **atomic sentences**; connectives combine them as before.
- **Quantifiers:** $\forall x\,\varphi$ (for all), $\exists x\,\varphi$ (there exists). A variable occurrence is **bound** if inside the scope of a quantifier on it, otherwise **free**. A sentence has no free variables.
- **Semantics:** a model has a non-empty **domain** of objects and an interpretation of every constant, function and predicate. Truth of $\forall x\,\varphi$: φ holds for *every* domain element; $\exists x\,\varphi$: for *at least one*.

Translating English: the two golden patterns

All S are P	$\forall x\,(S(x) \to P(x))$	(use $\to$ with $\forall$)
Some S is P	$\exists x\,(S(x) \wedge P(x))$	(use $\wedge$ with $\exists$)
No S is P	$\forall x\,(S(x) \to \neg P(x))$	$\equiv\ \neg\exists x\,(S(x) \wedge P(x))$
Not all S are P	$\exists x\,(S(x) \wedge \neg P(x))$	

Writing $\forall x\,(S(x) \wedge P(x))$ means *everything* is an S and a P; writing $\exists x\,(S(x) \to P(x))$ is true whenever anything fails to be an S: both are almost always wrong translations.

Quantifier rules

- **De Morgan for quantifiers:** $\neg\forall x\,\varphi \equiv \exists x\,\neg\varphi$; $\neg\exists x\,\varphi \equiv \forall x\,\neg\varphi$.
- Same-type quantifiers commute: $\forall x\forall y \equiv \forall y\forall x$, $\exists x\exists y \equiv \exists y\exists x$.
- **Mixed quantifiers do not commute:** $\exists y\,\forall x\,\varphi(x,y) \models \forall x\,\exists y\,\varphi(x,y)$, but not conversely.
- $\forall x(\varphi \wedge \psi) \equiv \forall x\,\varphi \wedge \forall x\,\psi$; $\exists x(\varphi \vee \psi) \equiv \exists x\,\varphi \vee \exists x\,\psi$. But $\forall x(\varphi \vee \psi) \not\equiv \forall x\,\varphi \vee \forall x\,\psi$.

Evaluating sentences in finite models (checked by program)

Domain $\{1,2,3\}$ with $<$:

- $\forall x\,\exists y\,(x < y)$ is **false** (the element 3 has nothing larger).
- $\exists x\,\forall y\,(x < y \ \vee\ x = y)$ is **true** (take $x = 1$, the minimum).
- $\forall x\,\exists y\,(y < x)$ is **false** (nothing is below 1).

Order of quantifiers: let Loves$= \{(1,2),(2,3),(3,3)\}$ on the domain $\{1,2,3\}$. $\forall x\,\exists y\,\text{Loves}(x,y)$ (everyone loves someone) is **true**; $\exists y\,\forall x\,\text{Loves}(x,y)$ (someone is loved by everyone) is **false** (nobody is loved by 1, 2 and 3 together).

6.1 Unification and resolution in first-order logic

Unification

A **substitution** θ replaces variables by terms. θ **unifies** two expressions if applying it makes them identical; the **most general unifier (MGU)** is the simplest such substitution. Algorithm: compare term by term; a variable is bound to the other term; two different constants (or function symbols) *fail*; the **occurs check** forbids binding a variable to a term that contains it.

Pair	Result
$P(x, f(y))$ and $P(a, f(z))$	$\{x/a,\ y/z\}$
Knows(John, x) and Knows(y, Mary)	$\{y/\text{John},\ x/\text{Mary}\}$
$Q(a, y)$ and $Q(x, g(x))$	$\{x/a,\ y/g(a)\}$
$P(x, x)$ and $P(a, b)$	**fails** (x cannot be both a and b)
$P(x, f(x))$ and $P(y, y)$	**fails** (occurs check: $y = x$ then $x = f(x)$)
x and $f(x)$	**fails** (occurs check)

Resolution for FOL: converting to clause form

(1) Eliminate $\leftrightarrow, \rightarrow$. (2) Move $\neg$ inward (also through quantifiers). (3) **Standardise variables** apart. (4) **Skolemise**: replace $\exists y$ by a Skolem constant or function of the universal variables in scope: $\exists x\,P(x) \rightarrow P(c)$; $\forall x\,\exists y\,\text{Loves}(x,y) \rightarrow \forall x\,\text{Loves}(x, f(x))$. (5) Drop the (now all universal) quantifiers. (6) Distribute $\vee$ over $\wedge$ to get clauses.
Then resolve clauses whose literals unify (with the MGU), using the refutation scheme as in the propositional case. Resolution is refutation-complete for FOL, but FOL entailment is only **semi-decidable**: a proof is found if the entailment holds, but the procedure may run forever if it does not.

Socrates

KB: $\forall x\,(\text{Human}(x) \to \text{Mortal}(x))$, Human(Socrates). Goal: Mortal(Socrates). Clauses: (1) $\neg\text{Human}(x) \vee \text{Mortal}(x)$; (2) Human(Socrates); (3) negated goal $\neg$Mortal(Socrates). Resolve (1) and (3) with $\{x/\text{Socrates}\}$: $\neg$Human(Socrates). Resolve with (2): the empty clause $\square$. Hence the goal follows. (This is generalised modus ponens.)

Common GATE traps

(1) With $\forall$ use $\to$; with $\exists$ use $\wedge$. (2) $\forall x\,\exists y$ is not the same as $\exists y\,\forall x$. (3) $\neg\forall x\,P(x)$ is $\exists x\,\neg P(x)$, not $\forall x\,\neg P(x)$. (4) Skolem functions depend on the universal variables in scope. (5) Unification must pass the occurs check. (6) Two distinct constants never unify. (7) First-order logic has quantification over *objects* only (second-order logic would quantify over predicates).

7 Reasoning under Uncertainty

Ramu & Somu: The wet-grass mystery

One morning Ramu finds the grass wet. "Did it rain?" he asks. Somu: "Maybe, or the sprinkler was on. But if I tell you the sprinkler *was* on, rain becomes less likely." Logic cannot say "less likely"; **probability** can. A **Bayesian network** draws who influences whom, so we can answer such questions without a huge table.

7.1 Probability toolkit

Rules you need

Product rule $P(a,b) = P(a \mid b)P(b)$; marginalisation $P(a) = \sum_b P(a,b)$; **Bayes** $P(a \mid b) = \dfrac{P(b \mid a)P(a)}{P(b)}$; chain rule $P(x_1,\ldots,x_n) = \prod_i P(x_i \mid x_1,\ldots,x_{i-1})$. $A \perp\!\!\!\perp B$ iff $P(A,B) = P(A)P(B)$; $A \perp\!\!\!\perp B \mid C$ iff $P(A,B \mid C) = P(A \mid C)P(B \mid C)$. A full joint over n Booleans needs $2^n - 1$ numbers.

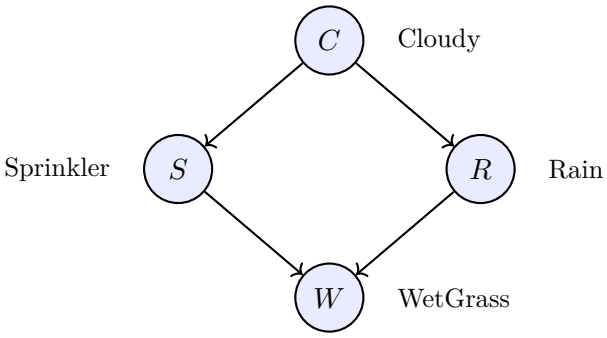
7.2 Bayesian networks

Definition

A **Bayesian network** is a directed acyclic graph (one node per variable) plus a conditional probability table (CPT) $P(X_i \mid \text{Parents}(X_i))$ per node. It represents the joint as

$$P(x_1,\ldots,x_n) = \prod_i P(x_i \mid \text{parents}(X_i)).$$

Each Boolean node with k parents needs 2^k numbers. **Local Markov property:** a node is independent of its non-descendants given its parents. **Markov blanket** of X: parents, children, and the children's other parents; given it, X is independent of everything else.



The running network

CPTs: $P(C) = 0.5$; $P(S \mid C) = 0.1$, $P(S \mid \neg C) = 0.5$; $P(R \mid C) = 0.8$, $P(R \mid \neg C) = 0.2$; $P(W \mid S, R) = 0.99,\ 0.9,\ 0.9,\ 0$ for $(S, R) = (1,1), (1,0), (0,1), (0,0)$.
Factorisation: $P(C, S, R, W) = P(C)\,P(S \mid C)\,P(R \mid C)\,P(W \mid S, R)$.
Parameters: $1 + 2 + 2 + 4 = \mathbf{9}$ numbers, against $2^4 - 1 = \mathbf{15}$ for the full joint (the saving grows enormously for sparse graphs).
One joint entry: $P(C{=}1, S{=}0, R{=}1, W{=}1) = 0.5 \cdot 0.9 \cdot 0.8 \cdot 0.9 = \mathbf{0.324}$.
Marginals (by summing the joint): $P(W) = 0.6471$, $P(R) = 0.5$, $P(S) = 0.3$.

7.3 Conditional independence and d-separation

d-separation: three basic structures

A path is **blocked** by a set Z if it contains…

Structure	Path	Blocked when
Chain	$A \to B \to C$	$B \in Z$
Fork (common cause)	$A \leftarrow B \to C$	$B \in Z$
Collider (common effect)	$A \to B \leftarrow C$	$B \notin Z$ **and no descendant of** B **in** Z

X and Y are **d-separated** by Z if every path between them is blocked; then $X \perp\!\!\!\perp Y \mid Z$ holds in every distribution that factorises over the graph. Observing a collider (or its descendant) *opens* the path.

Reading the network (numbers verified)

- $S \perp\!\!\!\perp R \mid C$: the path $S \leftarrow C \to R$ is a fork blocked by C. Check: $P(S{=}1 \mid C{=}1, R{=}1) = 0.1 = P(S{=}1 \mid C{=}1)$.
- $S \not\perp\!\!\!\perp R$ marginally (path through C is open): $P(S{=}1 \mid R{=}1) = 0.18 \neq P(S{=}1) = 0.30$.
- **Explaining away.** W is a collider between S and R. Given $W{=}1$: $P(R{=}1 \mid W{=}1) = 0.708$. Learn also that the sprinkler was on: $P(R{=}1 \mid W{=}1, S{=}1) = \mathbf{0.320}$. The sprinkler "explains away" the wet grass, so rain becomes less likely. Likewise $P(S{=}1 \mid W{=}1) = 0.430$ drops to 0.194 once $R{=}1$ is known.
- $C \perp\!\!\!\perp W \mid \{S, R\}$ (both paths are chains blocked by S and R).

7.4 Exact inference

Inference by enumeration

Query $P(X \mid e) \propto \sum_y P(X, e, y)$ over hidden variables y; normalise at the end. Correct but exponential: it recomputes shared sub-sums.

Variable elimination

Keep the CPTs as **factors**. **Algorithm:** (1) restrict every factor to the observed evidence; (2) for each hidden variable Z in a chosen **elimination order**: multiply all factors containing Z, then **sum Z out**, producing a new factor; (3) multiply the remaining factors, and normalise. Factors are reused, which avoids repeated work. The cost is exponential in the size of the largest factor created, which depends on the order; the best order's largest factor has size related to the graph's **treewidth**. Finding the optimal order is NP-hard (heuristics: min-degree, min-fill). Irrelevant (barren) nodes, those that are neither the query, evidence nor their ancestors, can simply be dropped. Exact inference in general networks is NP-hard (polynomial on polytrees).

Variable elimination for $P(R \mid W{=}1)$

Factors after restricting $W{=}1$: $P(C)$, $P(S \mid C)$, $P(R \mid C)$, $f(S, R) = P(W{=}1 \mid S, R)$. Eliminate S, then C.
Step 1. $f_5(C, R) = \sum_S P(S \mid C) f(S, R)$:

$$f_5(0,0) = 0.45, \quad f_5(0,1) = 0.945, \quad f_5(1,0) = 0.09, \quad f_5(1,1) = 0.909.$$

(e.g. $f_5(0,0) = 0.5 \cdot 0 + 0.5 \cdot 0.9 = 0.45$.)
Step 2. $f_6(R) = \sum_C P(C) P(R \mid C) f_5(C, R)$:

$$f_6(R{=}0) = 0.5 \cdot 0.8 \cdot 0.45 + 0.5 \cdot 0.2 \cdot 0.09 = 0.189,$$

$$f_6(R{=}1) = 0.5 \cdot 0.2 \cdot 0.945 + 0.5 \cdot 0.8 \cdot 0.909 = 0.4581.$$

Step 3. Normaliser $Z = 0.189 + 0.4581 = 0.6471 = P(W{=}1)$, so

$$P(R{=}1 \mid W{=}1) = \frac{0.4581}{0.6471} = \mathbf{0.708}.$$

Same answer as enumeration, with smaller intermediate tables (largest factor has 2 variables).

7.5 Approximate inference by sampling

Sampling methods

- **Prior (direct) sampling:** sample each variable in topological order from $P(X_i \mid \text{parents})$. Frequencies estimate probabilities; error shrinks like $1/\sqrt{N}$.
- **Rejection sampling:** draw prior samples, throw away those inconsistent with the evidence, estimate from the rest. Wasteful when the evidence is rare (the accepted fraction estimates $P(e)$).
- **Likelihood weighting:** fix the evidence variables and sample only the others; weight each sample by the product of $P(e_j \mid \text{parents})$. No rejections, but weights can become tiny when evidence is unlikely or deep in the network.
- **Gibbs sampling (MCMC):** start from any state consistent with the evidence; repeatedly

resample one non-evidence variable from $P(X \mid \text{Markov blanket})$. The samples form a Markov chain whose stationary distribution is the posterior; early samples (*burn-in*) are discarded and consecutive samples are correlated.

Estimating $P(R{=}1 \mid W{=}1)$ (simulated, $N = 200\,000$)

Method	Estimate
Exact (enumeration / VE)	0.7079
Rejection sampling	0.7083 (accepted fraction $0.6485 \approx P(W) = 0.6471$)
Likelihood weighting	0.7084
Gibbs sampling	0.7097

All converge to the exact value; prior sampling alone estimated $P(R) \approx 0.501$ (true 0.5).

Common GATE traps

(1) Arrows are about the factorisation, not necessarily causation. (2) Conditioning on a *collider* makes variables dependent; conditioning on a chain or fork *separates* them. (3) Independence is not transitive. (4) Rejection sampling discards samples; likelihood weighting does not but uses weights. (5) Gibbs needs a Markov blanket, and burn-in. (6) Elimination order affects cost, not the answer. (7) Remember to *normalise*.

8 One-page Cheat Sheet

Search

- BFS: complete, optimal for unit costs, $O(b^d)$ time *and* space. DFS: $O(bm)$ space, not optimal, incomplete on infinite/cyclic spaces. DLS/IDS: IDS is complete, optimal for unit cost, $O(b^d)$ time, $O(bd)$ space. UCS: expands by $g(n)$, optimal.
- Greedy: $f = h$ (not optimal). A*: $f = g + h$; optimal with admissible h (tree search) or consistent h (graph search). Consistent $\Rightarrow$ admissible, not conversely. Dominance: larger admissible h expands no more nodes.
- Minimax value by DFS; alpha-beta prunes when $\alpha \geq \beta$, same result, best case $O(b^{m/2})$.

Logic and uncertainty

- $KB \models \alpha \iff KB \wedge \neg\alpha$ unsatisfiable; n atoms $\Rightarrow 2^n$ models. Resolution: sound, refutation-complete. Forward chaining: linear for Horn.
- FOL: $\forall \rightarrow$, $\exists \wedge$; $\neg\forall \equiv \exists\neg$; Skolemise $\exists$; MGU with occurs check.
- BN: $P = \prod P(x_i \mid pa_i)$; d-separation (chain/fork blocked by observed middle; collider blocked unless observed); VE = restrict, multiply, sum out, normalise; sampling: prior, rejection, likelihood weighting, Gibbs.

9 Practice Questions

Answers are given right after each question.

1. BFS with branching factor b and shallowest goal at depth d: worst-case time? *Ans:* $O(b^d)$ (space too).
2. Which uninformed strategy is complete and has space linear in depth? *Ans:* none is both; IDS is complete with $O(bd)$ space (and optimal for unit costs).
3. Is DFS optimal? *Ans:* no (it may find a deeper goal first).
4. UCS reduces to BFS when…? *Ans:* all step costs are equal.
5. Define admissible heuristic. *Ans:* $h(n) \le h^*(n)$ for every n (never overestimates).
6. Define consistency. *Ans:* $h(n) \le c(n, n') + h(n')$ for every edge; implies admissibility.
7. $h_1 \le h_2$ both admissible. Which expands fewer nodes in A*? *Ans:* h_2 (dominates).
8. Greedy best-first uses $f =$? *Ans:* $h(n)$; it is not optimal.
9. A* with $h = 0$ is…? *Ans:* uniform-cost search.
10. Two admissible heuristics h_1, h_2: is $\max(h_1, h_2)$ admissible? *Ans:* yes (and dominates both). Is $h_1 + h_2$? *Ans:* not necessarily.
11. Is Manhattan distance admissible for the 8-puzzle? *Ans:* yes; so is the number of misplaced tiles, and Manhattan dominates it.
12. Best-case alpha-beta time with branching b, depth m? *Ans:* $O(b^{m/2})$.
13. Does alpha-beta change the minimax value of the root? *Ans:* no.
14. In minimax, who minimises? *Ans:* the opponent (MIN); the first player (MAX) maximises.
15. When is expectimax used? *Ans:* games with chance nodes; chance nodes take the expected value.
16. Number of Boolean functions of 3 variables? *Ans:* $2^8 = 256$.
17. Number of models of $p \vee q \vee r$ over 3 atoms? *Ans:* 7.
18. Number of models of $(p \vee q) \wedge (\neg p \vee r)$ over $\{p, q, r\}$? *Ans:* 4.
19. Is $(p \to q) \to (q \to p)$ valid? *Ans:* no.
20. Contrapositive of $p \to q$? *Ans:* $\neg q \to \neg p$ (equivalent).
21. Negate $\forall x\, \exists y\, \text{Loves}(x, y)$. *Ans:* $\exists x\, \forall y\, \neg\text{Loves}(x, y)$.
22. Translate "Every student likes some course". *Ans:* $\forall x\, (\text{St}(x) \to \exists y\, (\text{Co}(y) \wedge \text{Likes}(x, y)))$.
23. MGU of $\text{Knows}(\text{John}, x)$ and $\text{Knows}(y, \text{Mary})$? *Ans:* $\{y/\text{John},\ x/\text{Mary}\}$.
24. Does $P(x, x)$ unify with $P(a, b)$? *Ans:* no.
25. Does x unify with $f(x)$? *Ans:* no (occurs check).
26. Skolemise $\forall x\, \exists y\, \text{Loves}(x, y)$. *Ans:* $\forall x\, \text{Loves}(x, f(x))$.
27. Forward chaining with $A \wedge B \to C$, $C \to D$, facts A, B: derived set? *Ans:* $\{A, B, C, D\}$.
28. In the wet-grass network, parameters needed? *Ans:* 9 (vs 15).
29. $P(C{=}1, S{=}0, R{=}1, W{=}1)$? *Ans:* 0.324.
30. Are S and R independent given C? *Ans:* yes (fork blocked).

31. Are S and R independent given W? *Ans:* no (collider observed); explaining away.
32. $P(R{=}1 \mid W{=}1)$? *Ans:* 0.708.
33. $P(R{=}1 \mid W{=}1, S{=}1)$? *Ans:* 0.320.
34. Which sampling method rejects samples? *Ans:* rejection sampling.
35. Which method weights samples by evidence likelihood? *Ans:* likelihood weighting.
36. Gibbs sampling resamples a variable from its distribution given. . . ? *Ans:* its Markov blanket.
37. Is the order of variable elimination relevant for correctness? *Ans:* no, only for efficiency.

Idea in plain words

[How to study this section] Draw a search tree by hand for every algorithm until you can predict the expansion order. Practise translating English into logic with the two patterns until it is automatic. For networks, always write the factorisation first, then decide which variables to eliminate. Redo the explaining-away numbers once with pencil and paper: if you can reproduce 0.708 and 0.320, you understand the whole chapter.